PQC IoT SENTINEL

Consolidated Research Results, Discussion, Conclusion and Future Work

BL602 Post-Quantum Authenticated Application-Layer Security vs DTLS 1.2 Baseline

Research question
Can a post-quantum authenticated application-layer security architecture operate reliably on resource-constrained BL602 IoT nodes over extended periods, and what operational overhead does it introduce compared with conventional DTLS-secured CoAP?

Evidence finalized: 18 September 2026

Prepared as a thesis-ready technical synthesis of the completed experiments

Executive Summary

Core outcome. The experiments show that the implemented post-quantum authenticated application-layer security architecture can operate for approximately 24 hours on BL602-class constrained IoT hardware with very high end-to-end reliability and stable sampled runtime memory behavior.

Metric	PQC application-layer security	DTLS 1.2 baseline
Duration	23.999 h	24.000 h
Sender successes	32,790	42,346
Receiver successes	32,789	42,346
End-to-end gap	1	0
End-to-end match	99.9970%	100.0000%
Terminal sender failures	0	2
Successful messages/hour	1,366.31	1,764.42
Receiver linked flash	392.33 KiB	588.00 KiB
Sender linked flash	388.92 KiB	577.09 KiB
Receiver static RAM	104.81 KiB	91.90 KiB
Sender static RAM	101.22 KiB	91.90 KiB
Receiver max observed protocol-task stack use	10.28 KiB (24 h)	5.72 KiB (short characterization)

Interpretation in one sentence
The PQC design was operationally feasible and reliable on BL602, but its overhead was multidimensional: it used more static RAM and had lower observed system-level message throughput, while the evaluated complete firmware images were substantially smaller than the DTLS builds.

Abstract

This work evaluates the feasibility of a post-quantum authenticated application-layer security architecture on resource-constrained BL602 IoT nodes and compares its operational behavior with a conventional DTLS 1.2 secured-CoAP baseline. The PQC implementation combines a project-labeled ML-KEM-512-style key-establishment mechanism with HKDF-SHA-256, AES-256-CCM, per-device HMAC-SHA-256 authentication, receiver public-key fingerprint verification, persistent sequence/replay state, and authenticated acknowledgements. A 24-hour endurance experiment produced 32,790 authenticated sender deliveries and 32,789 receiver decryptions, corresponding to a 99.9970% end-to-end count match, with zero terminal sender, authentication, replay, or persistent-state failures. The DTLS baseline completed 42,346 successful sender and receiver exchanges with two sender message failures, zero handshake failures, and 143 successful handshakes. Resource analysis showed that the PQC receiver and sender required approximately 392.33 KiB and 388.92 KiB of linked flash, compared with 588.00 KiB and 577.09 KiB for DTLS, while PQC static RAM was approximately 10-14% higher. The PQC receiver maintained stable post-session free heap

throughout the 24-hour run and used a maximum observed 10.28 KiB of its configured 48 KiB protocol-task stack. DTLS achieved a higher observed message rate, but the protocol structures and message schedules differed, so this is not a direct cryptographic speed comparison. The results demonstrate that a post-quantum authenticated application-layer architecture is practically implementable on BL602-class constrained hardware, with measurable but non-prohibitive memory and throughput trade-offs.

1. Scope, Terminology and Evidence Basis

This report consolidates the completed PQC endurance experiment, the completed DTLS 1.2 endurance baseline, the finalized Splunk dashboard export, linked ELF/BIN resource measurements, and the separate DTLS runtime-memory characterization. It is written as a thesis-ready synthesis rather than as a deployment certification report.

- PQC implementation: custom application-layer CoAP/UDP security using a bundled Kyber512-style / project-labeled ML-KEM-512-style KEM, HKDF-SHA-256, AES-256-CCM, per-device HMAC-SHA-256 authentication, receiver fingerprint pinning, persistent replay/sequence state, and authenticated acknowledgements.
- Terminology constraint: the KEM should be described as ML-KEM-512-style or Kyber512-style unless formal standards conformance is independently validated. This report does not claim FIPS 203 certification or compliance.
- DTLS baseline: DTLS 1.2 with ECDHE P-256, ECDSA P-256, AES-256-CCM and CoAP over UDP port 5684.
- The DTLS development certificate material is laboratory fixture material, not deployment credentials. The baseline verifies server identity but does not provide an equivalent to the PQC per-device client HMAC mechanism, and DTLS HelloVerifyRequest cookies are disabled in the inherited SDK port.
- The two 24-hour experiments were conducted sequentially and used different protocol/message structures. Therefore, message-rate results are system-level observations, not isolated cryptographic benchmarks.

2. Experimental Design Summary

Item	Configuration
Hardware	Two PineCone BL602 boards, one sender and one receiver/gateway
UART	921600 baud
PQC transport	CoAP/UDP application-layer protected messages
DTLS transport	DTLS 1.2 secured CoAP over UDP/5684
PQC monitoring	Structured firmware events forwarded to Splunk
DTLS monitoring	Dedicated DTLS live bridge and Splunk event stream
PQC endurance ID	pqc-experiment
DTLS endurance ID	DTLS-experiment-final
DTLS sender cadence	Up to 300 messages per session, 2000 ms inter-message delay, repeated sessions
DTLS read timeout	5000 ms

2.1 Security pipeline context

The PQC path performs receiver public-key authentication, KEM/HKDF processing, AES-256-CCM protection, per-device HMAC authentication, replay/sequence checking, persistent state updates and authenticated ACK handling. The DTLS baseline places security in the DTLS record/session layer and amortizes handshake cost across multiple CoAP messages within a session.

3. Results

3.1 PQC 24-hour endurance result

The PQC experiment completed approximately 24 hours of protected communication. The sender recorded 32,790 authenticated deliveries and the receiver recorded 32,789 authenticated decryptions. The resulting count gap was one message and the final end-to-end count match was 99.9970%.

PQC metric	Observed value
Observed duration	23.999 h
Sender deliveries	32,790
Receiver decryptions	32,789
End-to-end gap	1
End-to-end match	99.9970%
Successful deliveries/hour	1,366.31
Sender retries	49
Duplicate ACK recovery events	194
Terminal sender failures	0
Authentication failures	0
Replay rejects	0
Persistent-state failures	0
HEC delivery failures	0
Invalid/rejected serial events	7

The capture bridge recorded 394,338 valid firmware events, and all 394,338 were queued and sent successfully to Splunk HEC. The experiment therefore provides both protocol-level evidence and monitoring-path health evidence.

3.2 DTLS 24-hour endurance result

The final DTLS run completed 24.000 hours. The sender and receiver each recorded 42,346 successful application exchanges. Two sender message attempts failed; the receiver recorded no failed messages. All 143 measured DTLS handshakes completed successfully.

DTLS metric	Observed value
Observed duration	24.000 h
Sender successes	42,346
Receiver successes	42,346
End-to-end gap	0
End-to-end match	100.0000%
Sender success rate	99.9953% (42,346 / 42,348)
Successful messages/hour	1,764.42
Sender message failures	2
Receiver message failures	0
Successful handshakes	143
Handshake failures	0
Sender session summaries	142

Diagnostic-count caution
The Splunk tile showing 66 DTLS failure/diagnostic events is a broad event count. It must not be reported as 66 failed application exchanges. The actual sender message-failure count is 2, with 0 handshake failures and 0 receiver message failures.

3.3 DTLS latency and handshake timing

Statistic	Per-message RTT	Sender handshake	Receiver handshake
Count	42,346	143	143
Mean	11.98 ms	1,926.68 ms	1,818.41 ms
Median	10.00 ms	1,743 ms	1,731 ms
P95	~18.1 ms	1,763.8 ms	1,746.0 ms
P99	~29.5 ms	1,858.5 ms	1,780.6 ms
Minimum	8 ms	1,722 ms	1,712 ms
Maximum	2,458 ms	27,712 ms	14,086 ms

The median and P95 values show that typical DTLS exchanges were much faster than the rare worst-case observations. The large maximum values therefore represent outliers rather than typical session behavior.

3.4 Firmware footprint and static RAM

Role	PQC linked flash	DTLS linked flash	DTLS increase vs PQC
Receiver	401,742 B / 392.33 KiB	602,110 B / 588.00 KiB	+49.87%
Sender	398,254 B / 388.92 KiB	590,938 B / 577.09 KiB	+48.38%

Role	PQC static RAM (data+bss)	DTLS static RAM	PQC increase vs DTLS
Receiver	107,324 B / 104.81 KiB	94,102 B / 91.90 KiB	+14.05%
Sender	103,652 B / 101.22 KiB	94,102 B / 91.90 KiB	+10.15%

Expressed relative to the DTLS image size, the PQC binary is approximately 33.28% smaller on the receiver and 32.61% smaller on the sender. This is a complete-build result, not a general claim that post-quantum cryptography always requires less flash.

3.5 Runtime memory behavior

Runtime metric	PQC receiver (24 h)	DTLS receiver (short run)	DTLS sender (short run)
Configured protocol-task stack	48.00 KiB	64.00 KiB	48.00 KiB
Minimum HWM	9,656 words	14,920 words	10,956 words
Max observed stack use	10.28 KiB	5.72 KiB	5.20 KiB
Stack utilization	21.42%	8.94%	10.84%
Post-session free heap	79,200 B	38,912 B	92,976 B
Minimum-ever free heap	70,864 B	33,072 B	49,744 B

For PQC, 32,788 valid post-session heap samples remained at 79,200 bytes, while 32,790 stack-HWM samples remained at 9,656 words. This provides evidence of stable sampled memory behavior throughout the 24-hour workload. DTLS runtime memory was measured separately over a short repeated-session characterization and is therefore not an equal-duration peak-memory comparison.

3.6 Wire-size context

Item	PQC observation	DTLS observation
Application plaintext	22 B	22 B
Protected/message representation	874 B protected PQC payload (firmware telemetry)	36 B CoAP message
DTLS record estimate	N/A	typically 73 B

Item	PQC observation	DTLS observation
DTLS-vs-app overhead	N/A	51 B relative to 22 B application payload

These numbers are not a direct full-packet comparison because they refer to different protocol layers: the PQC value is the protected application payload, while the DTLS value is an estimated DTLS record containing the CoAP message. They are included as protocol-size context rather than as a single normalized wire-overhead metric.

4. Discussion

4.1 Reliability and operational feasibility

The principal result is that the BL602 successfully sustained the full PQC application-security pipeline for approximately 24 hours. The final count match of 99.9970%, together with zero recorded terminal sender, authentication, replay and persistent-state failures, supports the feasibility of long-duration post-quantum authenticated communication on this class of device. The 49 retries and 194 duplicate-ACK recoveries are especially useful because they show operation beyond an idealized failure-free path: recovery mechanisms were exercised while the persistent sequence/replay controls remained intact.

The DTLS baseline was also highly reliable. Its 42,346 successful sender/receiver exchanges matched exactly, and all 143 handshakes succeeded. The two sender message timeouts therefore represent isolated recoverable events rather than general session instability.

4.2 Throughput must be interpreted as a system-level result

DTLS achieved approximately 29.14% more successful messages per hour than the PQC run (1,764.42 vs 1,366.31). It would be misleading to describe this as DTLS cryptography being 29% faster. DTLS amortizes a handshake over many application messages, whereas the PQC design performs an application-specific sequence of key, authentication, replay-state and acknowledgement operations. The message cadence, protocol exchanges and possibly wireless conditions were not identical. The result therefore quantifies observed system throughput under the tested configurations, not algorithm-only speed.

4.3 Flash and RAM trade-offs

The resource measurements show that post-quantum integration does not impose a uniform penalty across every resource dimension. The evaluated PQC images were roughly one-third smaller than the DTLS images, while their static RAM requirements were approximately 10-14% higher. The larger DTLS linked footprint is consistent with the inclusion of a complete DTLS/TLS state machine, X.509 certificate processing, ECDHE/ECDSA support and associated Mbed TLS functionality. Conversely, the PQC design maintains sizable cryptographic/protected-message buffers and application-specific authentication and persistence state, increasing static RAM pressure.

This result should remain implementation-specific. It does not establish that ML-KEM-class cryptography is generally smaller in flash than DTLS, nor that DTLS always has lower RAM requirements. It shows the resource composition of these two concrete BL602 firmware builds.

4.4 Runtime memory behavior

The 24-hour PQC receiver measurements are particularly valuable because post-session free heap and stack high-water mark remained stable over tens of thousands of samples. This is evidence against progressive resource depletion under the measured workload. It is more accurate to state that no progressive heap-loss or stack-growth trend was observed than to claim absolute proof of the absence of every possible memory leak, because short-lived allocations can occur between samples.

The short DTLS memory characterization also showed stable post-session free heap across repeated sessions. The sender minimum-ever heap low-water mark decreased by only 32 bytes and its stack high-water mark by 12 words (48

bytes) between the recorded clean sessions. This suggests repeatable session memory recovery in the observed short run. Equal-duration instrumentation would be required for a stronger peak-RAM comparison.

4.5 Security-model differences

The two designs should not be presented as security-semantic equivalents. The PQC application path combines receiver public-key fingerprint verification, per-device HMAC authentication, persistent replay/sequence protection and authenticated acknowledgements with post-quantum key establishment and AES-256-CCM. The DTLS baseline authenticates the server using development certificate fixtures but does not implement an equivalent per-device client HMAC or client-certificate authentication. DTLS HelloVerifyRequest cookies are also disabled in this inherited SDK configuration. Consequently, the DTLS run serves as a conventional secured-CoAP reference, not a perfectly feature-matched security replacement.

4.6 What the experiments demonstrate

- Post-quantum authenticated application-layer protection is implementable on BL602-class constrained hardware.
- Long-duration reliability remained high under the tested workload, including retry and duplicate-ACK recovery paths.
- Resource overhead is multidimensional: PQC used more static RAM and more observed receiver task stack, while DTLS used substantially more linked flash in these builds.
- DTLS delivered a higher observed system-level message rate, but the comparison is not a controlled cryptographic microbenchmark.
- The monitoring and evidence-preservation workflow was sufficiently robust to support reproducible analysis of the completed runs.

5. Limitations

1. The PQC and DTLS 24-hour experiments were sequential rather than simultaneous, so wireless conditions cannot be assumed identical.
2. The message cadence and protocol structures differed. Raw messages/hour is therefore not a pure cryptographic performance metric.
3. The PQC design is application-specific, whereas DTLS is a standardized transport-security protocol with certificate and protocol-state machinery. Complete firmware-size comparisons include these architectural differences.
4. The DTLS baseline used development certificate fixtures, did not implement an equivalent per-device client HMAC or client-certificate authentication, and ran without DTLS HelloVerifyRequest cookies.
5. PQC runtime memory was instrumented throughout the 24-hour run; DTLS runtime memory was collected later in a short three-session characterization. The runtime-memory comparison is therefore not equal-duration.
6. Power and energy consumption were not directly measured, so energy per protected message cannot be concluded.
7. The current KEM implementation should not be described as formally FIPS 203 compliant unless standards conformance is independently established.
8. The findings apply to the tested BL602 hardware, SDK, compiler/build configuration, firmware and network environment and should not be generalized uncritically to all constrained IoT platforms.

6. Direct Answer to the Research Question

Answer

Yes. Under the tested conditions, the post-quantum authenticated application-layer architecture operated for approximately 24 hours on BL602 nodes with 99.9970% end-to-end count correspondence, zero terminal sender/authentication/replay/persistent-state failures, and stable sampled receiver memory behavior. Its measured overhead relative to the DTLS baseline was not a single penalty: it used about 10-14% more static RAM and achieved a lower observed system-level message rate, but its complete linked firmware images were about 32-33% smaller. The result supports operational feasibility while identifying RAM, protocol-size and throughput trade-offs that must be considered in deployment design.

7. Conclusion

This research demonstrates that post-quantum authenticated protection on a small IoT microcontroller is not merely a desktop or simulation result. The implemented BL602 system executed the complete application-layer security path over an extended 24-hour experiment while maintaining high reliability, stable sampled memory behavior and fail-closed security controls. The conventional DTLS baseline likewise completed 24 hours with high reliability, providing a useful classical reference on the same hardware class.

The comparison reveals an engineering trade-off rather than a universal winner. DTLS provided higher observed message throughput and lower static RAM usage, while the PQC implementation required substantially less linked flash in the evaluated builds and incorporated additional application-specific mechanisms such as per-device HMAC authentication, persistent replay state and authenticated acknowledgements. The practical decision between these architectures therefore depends on the security model, RAM and flash constraints, communication cadence, recovery requirements and standards/deployment context.

8. Research Contributions

1. A working post-quantum authenticated application-layer security architecture implemented on physical BL602 IoT nodes.
2. A completed 24-hour endurance evaluation covering delivery, decryption, retries, authenticated ACK recovery, replay state, persistent state and monitoring health.
3. A completed 24-hour DTLS 1.2 / ECDHE P-256 / ECDSA P-256 / AES-256-CCM secured-CoAP reference run on the same device class.
4. A comparative resource analysis using linked ELF/BIN sizes, static RAM, runtime heap observations and task-stack high-water marks.
5. Structured telemetry and Splunk dashboards that separate PQC and DTLS evidence and preserve final-run counters and timing.
6. Frozen research archives with checksum verification for the final DTLS endurance and DTLS memory-characterization evidence.

9. Future Work

1. Instrument PQC and DTLS identically from the start and repeat both for equal-duration 24-hour runtime-memory measurements.
2. Measure current draw and energy per successful protected message during idle, key establishment, message processing and DTLS handshake phases.

3. Run a controlled equal-cadence comparison in which both systems send the same number and size of application messages under the same network impairment profile.
4. Perform controlled resilience experiments under packet loss, delay, jitter, disconnection, duplication and reordering.
5. Execute adversarial tests using replayed, modified, forged and incorrectly authenticated traffic and quantify rejection cost and fail-closed behavior.
6. Extend the DTLS reference with deployment-grade credentials, mutual authentication if required, and HelloVerifyRequest cookie support.
7. Evaluate the architecture on additional constrained microcontrollers to determine whether the BL602 resource trade-offs generalize.
8. Use and validate a standards-conformant ML-KEM implementation before making formal FIPS 203 compliance claims.

10. Evidence Preservation and Reproducibility Notes

Evidence item	Status / identifier
Final PQC 24-hour evidence	Archived under research_archive/PQC_24H_FINAL; manifest checks verified
Final DTLS 24-hour evidence	Archived under research_archive/DTLS_24H_FINAL
Final DTLS archive SHA-256	f01ec0f7f49e88c8750f46a6249f14283a7a62e02a56be77455a8b719669cebe
DTLS runtime-memory characterization	Archived under research_archive/DTLS_MEMORY_CHARACTERIZATION_20260918
DTLS memory archive SHA-256	0f89dbd7b23f622d5adf37d362562ac394b5e7d9de07c3a67de641b290fd5b8c
Final Splunk report	22-page export finalized 2026-09-18 18:19:33 CEST
Final DTLS run ID	DTLS-experiment-final
Final PQC experiment ID	pqc-experiment

The archive hashes above should be retained in the thesis research package so that the exact evidence bundles can be independently re-verified after transfer or backup.

11. Suggested Thesis Integration

For a conventional thesis structure, the material in this report can be integrated as follows:

Thesis section	Use from this report
Abstract	Use the Abstract section with minor formatting changes required by the university template.

Thesis section	Use from this report
Methodology / Experimental Setup	Use Sections 1 and 2 plus implementation details from the project documentation.
Results	Use Section 3 and the quantitative tables.
Discussion	Use Section 4; retain the caveats around throughput, runtime memory and security-model equivalence.
Limitations	Use Section 5.
Conclusion	Use Sections 6 and 7.
Contributions	Use Section 8 if the thesis template contains a contributions section.
Future Work	Use Section 9.
Appendix / Reproducibility	Use Section 10 and add archive manifests/scripts if required.

Recommended wording discipline

Do not write that PQC was universally faster, more secure, or superior to DTLS. Report the measured differences and the distinct security models. Also avoid calling receiver process timing DTLS decryption time, and avoid claiming formal ML-KEM/FIPS 203 compliance without independent validation.

End of consolidated research report